

1. Layer vs Tier & Dependency Inversion

A **layer** is a logical code organization within the same app, while a **tier** means physical deployment on separate servers. Classic N-tier made BLL depend on DAL. Applying Dependency Inversion (DIP) flips this so DAL depends on BLL, keeping core business logic independent of database details.

Source: [Microsoft Docs - Architectural Styles](#)

2. Wiring Problem

Since PL no longer references DAL directly, registering DAL services into DI requires either dynamic assembly loading via Reflection ([Assembly.Load](#)) or a BLL extension method. Reflection is preferred to keep BLL clean from registration setup.

Source: [Microsoft Docs - Dependency injection in ASP.NET Core](#)

3. [dotnet ef CLI Commands & MigrationsAssembly](#)

[dotnet ef](#) uses [--project DAL](#) to locate the [DbContext](#) and [--startup-project PL](#) to read configurations. [MigrationsAssembly](#) defines where migration files live. Moving existing migrations to DAL changes the file location without breaking the database's [__EFMigrationsHistory](#) table.

Source: [Microsoft Docs - EF Core Migrations in Separate Projects](#)

4. Centralized Exception Handling ([IExceptionHandler](#) & [ProblemDetails](#))

[IExceptionHandler](#) globally catches unhandled exceptions and converts them into standardized [ProblemDetails](#) JSON responses with proper HTTP status codes (e.g., 404, 409), avoiding duplicated try-catch blocks in endpoints.

Source: [Microsoft Docs - Handle errors in ASP.NET Core](#)

5. Repository Pattern on top of EF Core

- **Pro:** Simplifies unit testing by mocking interfaces without initializing a real [DbContext](#).
-
- **Con:** EF Core's [DbContext](#) and [DbSet](#) already implement Repository and Unit of Work, making an extra repository layer redundant.
-
- Source: [Microsoft Docs - Infrastructure layer architecture](#)

-

1. Unit Test vs Integration Test

A **unit test** verifies a single class/method in isolation using test doubles in milliseconds. An **integration test** checks if real modules (e.g., Service + Database) work correctly together.

Source: [Microsoft Docs - Unit testing best practices](#)

2. Test Doubles: Stub vs Mock vs Fake

- **Stub:** Returns canned fake data.
- **Mock:** Verifies interactions and expected method calls.
- **Fake:** A lightweight working implementation (e.g., [FakeTimeProvider](#) or in-memory list).

- Source: [Martin Fowler - Mocks Aren't Stubs](#)

3. xUnit Attributes: [\[Fact\]](#), [\[Theory\]](#), [\[InlineData\]](#)

[\[Fact\]](#) tests a single invariant scenario. [\[Theory\]](#) executes the same test logic multiple times using different parameters passed via [\[InlineData\(...\)\]](#).

Source: [xUnit.net Documentation](#)

4. [TimeProvider](#), [FakeTimeProvider](#), and [FakeLogger](#)

[TimeProvider](#) abstracts system time. [FakeTimeProvider](#) allows setting or fast-forwarding time in tests. [FakeLogger](#) captures logged messages in memory for assertions.

Source: [Microsoft Docs - TimeProvider and FakeTimeProvider](#)

5. Why EF Core Docs Advise Against InMemory Provider

The InMemory provider is not a relational database engine—it ignores foreign keys, constraints, and raw SQL. Tests passing on InMemory can still fail on real SQL Server.

Source: [Microsoft Docs - Testing EF Core Applications](#)

1. What Architecture Tests Are & What Problem They Solve

Architecture tests are automated tests that enforce project design rules and boundaries, preventing circular dependencies and layer boundary violations as the codebase grows.

Source: [NetArchTest GitHub Repository](#)

2. Checking Type Dependencies vs Project References

Project references only restrict dependencies at the assembly level. Checking type dependencies inspects individual classes and namespaces, catching hidden violations like reflection calls or transitive imports.

Source: [ArchUnitNET Documentation](#)

Part 5: Health Check for SQL Server

1. Liveness vs Readiness & Status Codes

- **Liveness** ([/health/live](#)): Checks if the app process is running without testing external dependencies.
- **Readiness** ([/health/ready](#)): Checks if the app can handle traffic, including SQL Server connectivity.
- **Status Codes**: 200 (Healthy) / 503 (Unhealthy).
- Source: [Microsoft Docs - Health checks in ASP.NET Core](#)

2. Three Ways to Check SQL Server

1. Custom [IHealthCheck](#) with manual [SqlConnection](#) ([SELECT 1](#)).
- 2.
3. Built-in [AddDbContextCheck<T>](#).
- 4.
5. Community package [AspNetCore.HealthChecks.SqlServer](#). Options 2 and 3 are standard for production.
- 6.
7. Source: [AspNetCore.Diagnostics.HealthChecks Repository](#)
- 8.

3. Security & Timeout Protection in Health Checks

Endpoints must never expose sensitive details (connection strings or stack traces). Always enforce a strict timeout (e.g., 3s) with a cancellation token to prevent hanging requests when SQL Server is down.

Source: [Microsoft Docs - Health checks response customization](#)

Part 6: Background Jobs with Hangfire

1. Background Jobs & Framework Comparison

Background jobs offload heavy or scheduled tasks from HTTP request threads. Unlike volatile in-memory [BackgroundService](#), tools like Hangfire save job state to SQL Server, support retries, and provide a monitoring dashboard.

Source: [Hangfire Official Documentation](#)

2. Hangfire Schema & Job Idempotency

Hangfire creates its own SQL tables separate from EF Core. Because jobs retry on failure, job logic must be **idempotent** (running twice produces the same result). Pass only simple primitive IDs as parameters.

Source: [Hangfire Best Practices - Idempotency](#)

3. Recurring Jobs & Cron Expressions

Recurring jobs use Cron expressions (e.g., `0 2 * * *` for 02:00 UTC). Hangfire uses distributed SQL locks to prevent duplicate execution across multiple instances.

Source: [Hangfire Recurring Jobs](#)